

TECHNICAL LEARNING FILE

DEEP-09



Algorithm Design Patterns

Reusable strategies for hard problems

FORMAT	LENGTH	ACCESS
INTENSIVE FILE	50 PAGES	OFFLINE

Edition 2 - original curriculum - editorial review recommended



FILE / ORIENTATION

Purpose

01 FILE GUIDANCE

This optional advanced guide does not gate the core learning path. Apply algorithm design patterns with explicit assumptions, tests, tradeoffs, and limitations.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



FILE / ORIENTATION

Prerequisites

01 FILE GUIDANCE

Complete the related core track or pass its diagnostic.

NOUS AI ACADEMY

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



FILE / ORIENTATION

Study Method

01 FILE GUIDANCE

For each unit, explain the concept, follow the workflow, reproduce the example, analyze a failure, and complete the applied lab. Record evidence locally.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



UNIT 01 / CONCEPT

Invariants: Concept

01 OBJECTIVE

Explain and apply invariants using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Invariants is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithm Design Patterns, the terms Invariants are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should invariants be used, and what observable evidence would make that choice defensible? Build a small local example that applies invariants, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Invariants in one precise sentence and name one assumption.
2. Create a two-step example of invariants and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / WORKFLOW

Invariants: Workflow

01 OBJECTIVE

Explain and apply invariants using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Invariants is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to invariants.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies invariants, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Invariants in one precise sentence and name one assumption.
2. Create a two-step example of invariants and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / WORKED EXAMPLE

Invariants: Worked Example

01 OBJECTIVE

Explain and apply invariants using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies invariants, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns invariants into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Invariants in one precise sentence and name one assumption.
2. Create a two-step example of invariants and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / FAILURE ANALYSIS

Invariants: Failure Analysis

01 OBJECTIVE

Explain and apply invariants using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how invariants can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to invariants. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Invariants in one precise sentence and name one assumption.
2. Create a two-step example of invariants and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / APPLIED LAB

Invariants: Applied Lab

01 OBJECTIVE

Explain and apply invariants using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of invariants into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies invariants, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Invariants in one precise sentence and name one assumption.
2. Create a two-step example of invariants and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / CONCEPT

Divide and Conquer: Concept

01 OBJECTIVE

Explain and apply divide and conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Divide and Conquer is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithm Design Patterns, the terms Divide, Conquer are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should divide and conquer be used, and what observable evidence would make that choice defensible? Build a small local example that applies divide and conquer, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Divide and Conquer in one precise sentence and name one assumption.
2. Create a two-step example of divide and conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / WORKFLOW

Divide and Conquer: Workflow

01 OBJECTIVE

Explain and apply divide and conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Divide and Conquer is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to divide and conquer.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies divide and conquer, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Divide and Conquer in one precise sentence and name one assumption.
2. Create a two-step example of divide and conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / WORKED EXAMPLE

Divide and Conquer: Worked Example

01 OBJECTIVE

Explain and apply divide and conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies divide and conquer, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns divide and conquer into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Divide and Conquer in one precise sentence and name one assumption.
2. Create a two-step example of divide and conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / FAILURE ANALYSIS

Divide and Conquer: Failure Analysis

01 OBJECTIVE

Explain and apply divide and conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how divide and conquer can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to divide and conquer. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Divide and Conquer in one precise sentence and name one assumption.
2. Create a two-step example of divide and conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / APPLIED LAB

Divide and Conquer: Applied Lab

01 OBJECTIVE

Explain and apply divide and conquer using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of divide and conquer into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies divide and conquer, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Divide and Conquer in one precise sentence and name one assumption.
2. Create a two-step example of divide and conquer and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / CONCEPT

Greedy Proofs: Concept

01 OBJECTIVE

Explain and apply greedy proofs using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Greedy Proofs is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithm Design Patterns, the terms Greedy, Proofs are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should greedy proofs be used, and what observable evidence would make that choice defensible? Build a small local example that applies greedy proofs, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Greedy Proofs in one precise sentence and name one assumption.
2. Create a two-step example of greedy proofs and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / WORKFLOW

Greedy Proofs: Workflow

01 OBJECTIVE

Explain and apply greedy proofs using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Greedy Proofs is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to greedy proofs.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies greedy proofs, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Greedy Proofs in one precise sentence and name one assumption.
2. Create a two-step example of greedy proofs and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / WORKED EXAMPLE

Greedy Proofs: Worked Example

01 OBJECTIVE

Explain and apply greedy proofs using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies greedy proofs, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns greedy proofs into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Greedy Proofs in one precise sentence and name one assumption.
2. Create a two-step example of greedy proofs and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / FAILURE ANALYSIS

Greedy Proofs: Failure Analysis

01 OBJECTIVE

Explain and apply greedy proofs using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how greedy proofs can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to greedy proofs. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Greedy Proofs in one precise sentence and name one assumption.
2. Create a two-step example of greedy proofs and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / APPLIED LAB

Greedy Proofs: Applied Lab

01 OBJECTIVE

Explain and apply greedy proofs using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of greedy proofs into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies greedy proofs, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Greedy Proofs in one precise sentence and name one assumption.
2. Create a two-step example of greedy proofs and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / CONCEPT

Dynamic Programming: Concept

01 OBJECTIVE

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Dynamic Programming is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithm Design Patterns, the terms Dynamic, Programming are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should dynamic programming be used, and what observable evidence would make that choice defensible? Build a small local example that applies dynamic programming, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / WORKFLOW

Dynamic Programming: Workflow

01 OBJECTIVE

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Dynamic Programming is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to dynamic programming.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies dynamic programming, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / WORKED EXAMPLE

Dynamic Programming: Worked Example

01 OBJECTIVE

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies dynamic programming, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns dynamic programming into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / FAILURE ANALYSIS

Dynamic Programming: Failure Analysis

01 OBJECTIVE

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how dynamic programming can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to dynamic programming. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / APPLIED LAB

Dynamic Programming: Applied Lab

01 OBJECTIVE

Explain and apply dynamic programming using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of dynamic programming into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies dynamic programming, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Dynamic Programming in one precise sentence and name one assumption.
2. Create a two-step example of dynamic programming and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / CONCEPT

Backtracking: Concept

01 OBJECTIVE

Explain and apply backtracking using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Backtracking is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithm Design Patterns, the terms Backtracking are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should backtracking be used, and what observable evidence would make that choice defensible? Build a small local example that applies backtracking, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Backtracking in one precise sentence and name one assumption.
2. Create a two-step example of backtracking and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / WORKFLOW

Backtracking: Workflow

01 OBJECTIVE

Explain and apply backtracking using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Backtracking is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to backtracking.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies backtracking, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Backtracking in one precise sentence and name one assumption.
2. Create a two-step example of backtracking and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / WORKED EXAMPLE

Backtracking: Worked Example

01 OBJECTIVE

Explain and apply backtracking using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies backtracking, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns backtracking into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Backtracking in one precise sentence and name one assumption.
2. Create a two-step example of backtracking and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / FAILURE ANALYSIS

Backtracking: Failure Analysis

01 OBJECTIVE

Explain and apply backtracking using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how backtracking can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to backtracking. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Backtracking in one precise sentence and name one assumption.
2. Create a two-step example of backtracking and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / APPLIED LAB

Backtracking: Applied Lab

01 OBJECTIVE

Explain and apply backtracking using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of backtracking into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies backtracking, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Backtracking in one precise sentence and name one assumption.
2. Create a two-step example of backtracking and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / CONCEPT

Graph Search: Concept

01 OBJECTIVE

Explain and apply graph search using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Graph Search is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithm Design Patterns, the terms Graph, Search are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should graph search be used, and what observable evidence would make that choice defensible? Build a small local example that applies graph search, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Graph Search in one precise sentence and name one assumption.
2. Create a two-step example of graph search and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / WORKFLOW

Graph Search: Workflow

01 OBJECTIVE

Explain and apply graph search using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Graph Search is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to graph search.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies graph search, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Graph Search in one precise sentence and name one assumption.
2. Create a two-step example of graph search and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / WORKED EXAMPLE

Graph Search: Worked Example

01 OBJECTIVE

Explain and apply graph search using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies graph search, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns graph search into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Graph Search in one precise sentence and name one assumption.
2. Create a two-step example of graph search and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / FAILURE ANALYSIS

Graph Search: Failure Analysis

01 OBJECTIVE

Explain and apply graph search using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how graph search can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to graph search. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Graph Search in one precise sentence and name one assumption.
2. Create a two-step example of graph search and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / APPLIED LAB

Graph Search: Applied Lab

01 OBJECTIVE

Explain and apply graph search using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of graph search into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies graph search, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Graph Search in one precise sentence and name one assumption.
2. Create a two-step example of graph search and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / CONCEPT

Randomized Methods: Concept

01 OBJECTIVE

Explain and apply randomized methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Randomized Methods is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithm Design Patterns, the terms Randomized, Methods are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should randomized methods be used, and what observable evidence would make that choice defensible? Build a small local example that applies randomized methods, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Randomized Methods in one precise sentence and name one assumption.
2. Create a two-step example of randomized methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / WORKFLOW

Randomized Methods: Workflow

01 OBJECTIVE

Explain and apply randomized methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Randomized Methods is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to randomized methods.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies randomized methods, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Randomized Methods in one precise sentence and name one assumption.
2. Create a two-step example of randomized methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / WORKED EXAMPLE

Randomized Methods: Worked Example

01 OBJECTIVE

Explain and apply randomized methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies randomized methods, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns randomized methods into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Randomized Methods in one precise sentence and name one assumption.
2. Create a two-step example of randomized methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / FAILURE ANALYSIS

Randomized Methods: Failure Analysis

01 OBJECTIVE

Explain and apply randomized methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how randomized methods can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to randomized methods. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Randomized Methods in one precise sentence and name one assumption.
2. Create a two-step example of randomized methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / APPLIED LAB

Randomized Methods: Applied Lab

01 OBJECTIVE

Explain and apply randomized methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of randomized methods into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies randomized methods, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Randomized Methods in one precise sentence and name one assumption.
2. Create a two-step example of randomized methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / CONCEPT

Approximation: Concept

01 OBJECTIVE

Explain and apply approximation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Approximation is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithm Design Patterns, the terms Approximation are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should approximation be used, and what observable evidence would make that choice defensible? Build a small local example that applies approximation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):
  queue <- [start]
  seen <- {start}
  while queue is not empty:
    node <- remove_front(queue)
    for each neighbor of node:
      if neighbor not in seen:
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Approximation in one precise sentence and name one assumption.
2. Create a two-step example of approximation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / WORKFLOW

Approximation: Workflow

01 OBJECTIVE

Explain and apply approximation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Approximation is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to approximation.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies approximation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Approximation in one precise sentence and name one assumption.
2. Create a two-step example of approximation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / WORKED EXAMPLE

Approximation: Worked Example

01 OBJECTIVE

Explain and apply approximation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies approximation, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns approximation into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Approximation in one precise sentence and name one assumption.
2. Create a two-step example of approximation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / FAILURE ANALYSIS

Approximation: Failure Analysis

01 OBJECTIVE

Explain and apply approximation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how approximation can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to approximation. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Approximation in one precise sentence and name one assumption.
2. Create a two-step example of approximation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / APPLIED LAB

Approximation: Applied Lab

01 OBJECTIVE

Explain and apply approximation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of approximation into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies approximation, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Approximation in one precise sentence and name one assumption.
2. Create a two-step example of approximation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / CONCEPT

Benchmark Design: Concept

01 OBJECTIVE

Explain and apply benchmark design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Benchmark Design is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Algorithm Design Patterns, the terms Benchmark, Design are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should benchmark design be used, and what observable evidence would make that choice defensible? Build a small local example that applies benchmark design, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Benchmark Design in one precise sentence and name one assumption.
2. Create a two-step example of benchmark design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / WORKFLOW

Benchmark Design: Workflow

01 OBJECTIVE

Explain and apply benchmark design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Benchmark Design is a central part of algorithm design patterns because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to benchmark design.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies benchmark design, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Benchmark Design in one precise sentence and name one assumption.
2. Create a two-step example of benchmark design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / WORKED EXAMPLE

Benchmark Design: Worked Example

01 OBJECTIVE

Explain and apply benchmark design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies benchmark design, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns benchmark design into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Benchmark Design in one precise sentence and name one assumption.
2. Create a two-step example of benchmark design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / FAILURE ANALYSIS

Benchmark Design: Failure Analysis

01 OBJECTIVE

Explain and apply benchmark design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how benchmark design can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to benchmark design. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Benchmark Design in one precise sentence and name one assumption.
2. Create a two-step example of benchmark design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / APPLIED LAB

Benchmark Design: Applied Lab

01 OBJECTIVE

Explain and apply benchmark design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of benchmark design into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies benchmark design, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
procedure BFS(start):  
  queue <- [start]  
  seen <- {start}  
  while queue is not empty:  
    node <- remove_front(queue)  
    for each neighbor of node:  
      if neighbor not in seen:  
        add neighbor to seen and queue
```

05 PRACTICE

1. Define Benchmark Design in one precise sentence and name one assumption.
2. Create a two-step example of benchmark design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



FILE / ORIENTATION

Deep-Dive Capstone

01 FILE GUIDANCE

Build a compact, reproducible artifact demonstrating algorithm design patterns. Include assumptions, a baseline, tests, failure analysis, results, limitations, and instructions for repeating the work offline.

NOUS AI ACADEMY

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.